



Department of
Software Engineering
BAHRIA UNIVERSITY
Discovering Knowledge

CSC- 321 DESIGN AND ANALYSIS OF ALGORITHMS

COURSE INSTRUCTOR

Dr. Qamaruddin Memon

Email: Qamaruddin.bukc@bahria.edu.pk

LECTURE 2a

Growth of Function

OUTLINE

- Properties Of Algorithm
- Algorithm Analysis
- Correctness
- Space Complexity
- How to find the Space Complexity
- Time Complexity
- How to find the Time Complexity

PROPERTIES OF AN ALGORITHM

Input -An algorithm has input values from a specified set.

Output -From each set of input values an algorithm produces output values from a specified set. The output values are the solution to the problem.

Definiteness -The steps of an algorithm must be defined precisely.

Correctness -An algorithm should produce the correct output values for each set of input values.

PROPERTIES OF AN ALGORITHM

Finiteness -An algorithm should produce the desired output after a finite (but perhaps large) number of steps for any input in the set.

Effectiveness -It must be possible to perform each step of an algorithm exactly and in a finite amount of time.

Generality -The procedure should be applicable for all problems of the desired form, not just for a particular set of input values

ALGORITHM ANALYSIS

The task of determining how much computing time and storage and algorithm requires.

- theoretical analysis (Mathematical)
- empirical analysis

How good is the algorithm?

- Correctness
- Time efficiency
- Space efficiency

it needs mathematical skills.

Best case, average case, and worst case.

CORRECTNESS

- An algorithm is said to be correct if, for every input instance, it halts with the correct output. A correct algorithm solves the given computational problem.
- Partial correctness requires that if an answer is returned it will be correct.
Total correctness additionally requires that the algorithm terminates.
- An incorrect algorithm might not halt at all on some input instances, or it might halt with other than the

SPACE COMPLEXITY

Space complexity is the amount of memory used by the algorithm (including the input values to the algorithm) to execute and produce the result.

Algorithm uses memory space for three reasons

- Instruction Space
- Environmental Stack
- Data Space
- **Memory used by different data types**

How much memory resource, algorithm will require?

We usually consider only Data Space for space complexity

HOW TO CALCULATE SPACE COMPLEXITY

Formula:

Space Complexity = Auxiliary Space + Input space

Type	Size
bool, char, unsigned char, signed char, __int8	1 byte
__int16, short, unsigned short, wchar_t, __wchar_t	2 bytes
float, __int32, int, unsigned int, long, unsigned long	4 bytes
double, __int64, long double, long long	8 bytes

HOW TO CALCULATE SPACE COMPLEXITY

Example 1

```
{  
    int z = a + b + c;  
    return(z);  
}
```

Solution:

In the above expression, variables a, b, c and z are all integer types, hence they will take up 4 bytes each, so total memory requirement will be $(4(4) + 4) = 20$ bytes, this additional 4 bytes is for return value.

HOW TO CALCULATE SPACE COMPLEXITY

Example 2

// n is the length of array a[]

```
int sum(int a[], int n)
```

```
{
```

```
    int x = 0;           // 4 bytes for x
```

```
    for(int i = 0; i < n; i++)    // 4 bytes for i
```

```
    {
```

```
        x = x + a[i];
```

```
    }
```

```
    return(x);
```

```
}
```

HOW TO CALCULATE SPACE COMPLEXITY

Solution:

- In the above code, $4*n$ bytes of space is required for the array `a[]` elements.
- 4 bytes each for `x`, `n`, `i` and the return value.

Hence the total memory requirement will be $(4n + 12)$, which is increasing linearly with the increase in the input value `n`, hence it is called as Linear Space Complexity.

TIME COMPLEXITY

Time complexity of an algorithm signifies the total time required by the program to run till its completion.

Time efficiency is analyzed by determining the number of repetitions of the basic operation as a function of input size.

Basic operation: the operation that contributes most towards the running time of the algorithm

Time Efficiency is speed. How long an algorithm takes to produce its result? We usually use the worst-case time complexity

GENERAL RULES

- Sequence of statements
 - the time for each statement is constant and the total time is also constant: $O(1)$
- if-then-else statements
 - the time for the whole if-then-else statement would be worst of both
- Loop statements
 - The loop executes N times, the sequence of statements also executes N times
- Nested loops
 - If inner loop is independent of the value of the outer loop's index then loop executes $N*M$ times
 - Other wise inner loop also executes N times so $N*N$

HOW TO FIND THE TIME COMPLEXITY

① Swap (a, b)

{ temp = a — 1

a = b — 1

b = temp — 1

}

$$f(n) = 1 + 1 + 1$$

$$f(n) = 3 \text{ (is constant)}$$

$$\text{Time Complexity} = O(1)$$

HOW TO FIND THE TIME COMPLEXITY

```
② Add (A, n)
{
  S = 0      — 1
  for (i = 0 ; i < n ; i++) — n + 1
  {
    S = S + A[i]; — n
  }
  return S; — 1
}
```

$b(n) = 1 + n + 1 + n + 1$
 $b(n) = 3 + 2n$
 Time Complexity = $O(n)$

HOW TO FIND THE TIME COMPLEXITY

③ Add (A, B, n)

```

{ for (i = 0; i < n; i++) — n+1
  { for (j = 0; j < n; j++) — n x (n+1)
    { C[i, j] = A[i, j] + B[i, j] — n x n
  }
}

```

$$b(n) = n + 1 + n \times (n + 1) + n^2$$

$$b(n) = n + 1 + n^2 + n + n^2$$

$$b(n) = 2n^2 + 2n + 1$$

Time Complexity = $O(n^2)$

HOW TO FIND THE TIME COMPLEXITY

④ Multiply (A, B, n)

```

{
  for (i = 0; i < n; i++) — n + 1
  {
    for (j = 0; j < n; j++) — n × (n + 1)
    {
      C[i, j] = 0 — n × n
      for (k = 0; k < n; k++) — n × n × (n + 1)
      {
        C[i, j] = C[i, j] + A[i, k] * B[k, j]
        ↓
        n × n × n
      }
    }
  }
}

```

$f(n) = 2n^3 + 3n^2 + 2n + 1$
 Time Complexity = $O(n^3)$

HOW TO FIND THE TIME COMPLEXITY

⑤ for ($i=0$; $i < n$; $i++$) — $n+1$
 {
 statement ; — n
 }
Time Complexity = $O(n)$

HOW TO FIND THE TIME COMPLEXITY

⑥ for (i = 0 ; i > 0 ; i--) — $n+1$
 {
 statement ; — n
 }
 Time Complexity = $O(n)$

HOW TO FIND THE TIME COMPLEXITY

⑦ for ($i=1; i \leq n; i=i+2$)
{ statement ; }
}

Time Complexity = $O(n)$

$\therefore \frac{n}{2}$ or even or $\frac{n}{20}$

HOW TO FIND THE TIME COMPLEXITY

⑧

```
for(i=0; i < n; i++)  
{  
  for(j=0; j < i; j++)  
  {  
    statement;  
  }  
}
```

i	j	no. of time
0	0 x	0
1	0 ✓	
	1 x	1
2	0 ✓	
	1 ✓	2
	2 x	

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$
$$= \frac{n^2 + n}{2}$$

$$\text{Time Complexity} = O(n^2)$$

HOW TO FIND THE TIME COMPLEXITY

⑩ for ($i=1; i \leq n; i=i*2$)
 { statement; }

Assume,

$$i \geq n$$

$$i = 2^k$$

$$2^k \geq n$$

$$2^k = n$$

$$k = \log_2 n$$

$$\begin{array}{l} i \\ 1 \\ 1 \times 2 = 2 \quad 2^1 \\ 2 \times 2 = 2^2 \\ 2^2 \times 2 = 8 \rightarrow 2^3 \\ \vdots \\ 2^k \end{array}$$

$$\text{Time Complexity} = O(\log_2 n)$$

HOW TO FIND THE TIME COMPLEXITY

⑪ for ($i = n$; $i \geq 1$; $i = i/2$)
{ statement ; }

Assume,

$$i \geq 1$$

$$\therefore \frac{n}{2^k} \geq 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$\frac{n}{2} + \frac{n}{2} + \frac{n}{2} + \dots + \frac{n}{2^k}$$

$$\text{Time Complexity} \\ = O(\log_2 n)$$

HOW TO FIND THE TIME COMPLEXITY

⑫ for ($i=0 ; i*i < n ; i++$)
{ statement ; }

Assume,

$$i * i < n$$

$$i * i \geq n$$

$$i^2 = n$$

$$i = \sqrt{n}$$

Time Complexity = $O(\sqrt{n})$

HOW TO FIND THE TIME COMPLEXITY

⑬ $\text{for } (i = 1 ; i < n ; i = i * 2) \text{ — } \log n$
 $\{ \quad P++ ; \quad \}$
 $\text{for } (j = 1 ; j < P ; j = j * 2) \text{ — } \log \log P$
 $\{ \quad \text{statement} ; \quad \}$
 Time Complexity = $O(\log \log n)$

HOW TO FIND THE TIME COMPLEXITY

⑭ for ($i = 0 ; i < n ; i++$) : — n
 {
 for ($j = 1 ; j < n ; j = j * 2$) — $n \times \log n$
 {
 Statement; — $n \times \log n$
 }
 }

 $f(n) = n + 2n \log n$
 Time Complexity = $O(n \log n)$

HOW TO FIND THE TIME COMPLEXITY

is

```
i = 0           - 1
while (i < n)    - n+1
{
    statement;   - n
    i++;         - n
}
```

Time Complexity = $O(n)$

HOW TO FIND THE TIME COMPLEXITY

⑩

```
i = n
while (i > 1)
{
    statement;
    i = i / 2;
}
```

Time Complexity = $O(\log n)$

HOW TO FIND THE TIME COMPLEXITY

(17)

```
a = 1
while (a < b)
{
    statement;
    a = a * 2;
}
```

Assume,

$$a \geq b$$

$$\therefore a = 2^k$$

$$2^k \geq b$$

$$2^k = b$$

$$k = \log_2 b$$

Time Complexity = $O(\log_2 n)$

a	
1	2^0
$1 \times 2 = 2$	2^1
$2 \times 2 = 4$	2^2
$4 \times 2 = 8$	2^3
$\vdots$	
2^k	

HOW TO FIND THE TIME COMPLEXITY

Case 1:
(18)

```

i = 1
k = 1
while (k < n)
{
    statement;
    k = k + i;
    i = i + 1;
}

```

Assume,

$$\begin{aligned}
 k &\geq n \\
 \frac{n(n+1)}{2} &\geq n \\
 n^2 &\geq n \\
 n &= \sqrt{n}
 \end{aligned}$$

i	k
1	1
2	1+1 = 2
3	2+2 = 4
4	2+2+3
5	2+2+3+4
⋮	⋮
m	m times

$$\begin{aligned}
 &1 + 2 + 3 + \dots + m \\
 \Rightarrow &\frac{m(m+1)}{2}
 \end{aligned}$$

Time Complexity = $O(\sqrt{n})$

HOW TO FIND THE TIME COMPLEXITY

Case 3:
(19)

```
while (m != n)
{
    if (m > n)
    {
        m = m - n;
    }
    else
    {
        n = n - m;
    }
}
```

Time Complexity = $O(n)$

Growth Of Function

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 \dots$$
$$< 2^n < 3^n < n^n$$

The Execution Time of Algorithms

- ▣ Each operation in an algorithm (or a program) has a cost.
 - ➔ Each operation takes a certain of time.

`count = count + 1;` ➔ take a certain amount of time, but it is constant

A sequence of operations:

`count = count + 1;`

Cost: c_1

`sum = sum + count;`

Cost: c_2

➔ Total Cost = $c_1 + c_2$

The Execution Time of Algorithms (cont.)

Example: Simple If-Statement

	<u>Cost</u>	<u>Times</u>
if (n < 0)	c1	1
absval = -n	c2	1
else		
absval = n;	c3	1

Total Cost $\leq c1 + \max(c2, c3)$

The Execution Time of Algorithms (cont.)

Example: Simple Loop

<u>Times</u>	<u>Cost</u>	
i = 1;	c1	
1		
sum = 0;	c2	
1		
while (i <= n) {	c3	n+1
i = i + 1;	c4	n
sum = sum + i;	c5	n
}		

$$\text{Total Cost} = c1 + c2 + (n+1)*c3 + n*c4 + n*c5$$

➔ The time required for this algorithm is proportional to n

The Execution Time of Algorithms (cont.)

Example: Nested Loop

	<u>Cost</u>	<u>Times</u>
<code>i=1;</code>	c1	1
<code>sum = 0;</code>	c2	1
<code>while (i <= n) {</code>	c3	n+1
<code>j=1;</code>	c4	n
<code>while (j <= n) {</code>	c5	n*(n+1)
<code>sum = sum + i;</code>	c6	n*n
<code>j = j + 1;</code>	c7	n*n
<code>}</code>		
<code>i = i + 1;</code>	c8	n
<code>}</code>		

Total Cost = $c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8$

➔ The time required for this algorithm is proportional to n^2

Some Mathematical Facts

□_n Some mathematical equalities are:

$$\sum_{i=1}^n 1 = 1 + 1 + \dots + 1 = n$$

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n * (n + 1)}{2} \approx \frac{n^2}{2}$$

$$\sum_{i=1}^n i^2 = 1 + 4 + \dots + n^2 = \frac{n * (n + 1) * (2n + 1)}{6} \approx \frac{n^3}{3}$$

$$\sum_{i=1}^n i^3 = 1 + 8 + \dots + n^3 = \left(\frac{n(n + 1)}{2} \right)^2$$

$$\sum_{i=0}^{n-1} 2^i = 0 + 1 + 2 + \dots + 2^{n-1} = 2^n - 1$$

Growth-Rate Functions – Example 1

<u>Times</u>	<u>Cost</u>		
<code>i = 1;</code>	<code>c1</code>		<code>1</code>
<code>sum = 0;</code>	<code>c2</code>		<code>1</code>
<code>while (i <= n) {</code>	<code>c3</code>		<code>n+1</code>
<code>i = i + 1;</code>	<code>c4</code>	<code>n</code>	
<code>sum = sum + i;</code>	<code>c5</code>	<code>n</code>	
<code>}</code>			

$$\begin{aligned}
 T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*c5 \\
 &= (c3+c4+c5)*n + (c1+c2+c3) \\
 &= a*n + b
 \end{aligned}$$

➔ So, the growth-rate function for this algorithm is **O(n)**

Growth-Rate Functions – Example2

	<u>Cost</u>	<u>Times</u>
i=1;	c1	1
sum = 0;	c2	1
while (i <= n) {	c3	n+1
j=1;	c4	n
while (j <= n) {	c5	n*(n+1)
sum = sum + i;	c6	n*n
j = j + 1;	c7	n*n
}		
i = i + 1;	c8	n
}		

$$\begin{aligned}
 T(n) &= c1 + c2 + (n+1)*c3 + n*c4 + n*(n+1)*c5 + n*n*c6 + n*n*c7 + n*c8 \\
 &= (c5+c6+c7)*n^2 + (c3+c4+c5+c8)*n + (c1+c2+c3) \\
 &= a*n^2 + b*n + c
 \end{aligned}$$

→ So, the growth-rate function for this algorithm is $O(n^2)$

Growth-Rate Functions – Example 3

	<u>Cost</u>	<u>Times</u>
• for (i=1; i<=n; i++)	c1	n+1
for (j=1; j<=i; j++)	c2	$\sum_{j=1}^n (j+1)$
for (k=1; k<=j; k++)	c3	$\sum_{j=1}^n \sum_{k=1}^j (k+1)$
x=x+1;	c4	$\sum_{j=1}^n \sum_{k=1}^j k$

$$\begin{aligned}
 T(n) &= c1*(n+1) + c2*\left(\sum_{j=1}^n (j+1)\right) + c3*\left(\sum_{j=1}^n \sum_{k=1}^j (k+1)\right) + c4*\left(\sum_{j=1}^n \sum_{k=1}^j k\right) \\
 &= c1n + c1 + c2n^2 + c2n + c3n^3 + (c3 + c4)2n^2 \\
 &= (c1 + c2)n + (c2 + 2c3 + 2c4)n^2 + c3n^3 \\
 &= a*n^3 + b*n^2 + c*n + d
 \end{aligned}$$

➔ So, the growth-rate function for this algorithm is **$O(n^3)$**

Sequential Search

```
int sequentialSearch(const int a[], int item, int n){  
    for (i = 0; i < n && a[i] != item; i++);  
    if (i == n)  
        return -1;  
    return i;  
}
```

Unsuccessful Search: $\rightarrow O(n)$

Successful Search:

Best-Case: *item* is in the first location of the array $\rightarrow O(1)$

Worst-Case: *item* is in the last location of the array $\rightarrow O(n)$

Average-Case: The number of key comparisons 1, 2, ..., n

$$\frac{\sum_{i=1}^n i}{n} = \frac{(n^2 + n)/2}{n} \quad \rightarrow O(n)$$

Binary Search-Iterative

```
int binarySearch(int a[], int size, int x) {  
    int low = 0;  
    int high = size - 1;  
    int mid;    // mid will be the index of  
                // target when it's found.  
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (a[mid] == x)  
            return mid;  
        else if (a[mid] > x)  
            high = mid - 1;  
        else  
            low = mid + 1;  
    }  
    return -1;  
}
```

Binary Search-Recursive

```
int binarySearch(int []a, int lo, int hi, int x)
{
    int mid= (lo+hi)/2;
    if(a[mid]==x) return mid;
    else if(a[mid]>x)
        return binarySearch(a,lo,mid-1);
    else return binarySearch(a mid+1,hi);
}
```

Binary Search – Analysis

$$T(n) = T(n/2) + c$$

$$T(1) = c$$

$$\begin{aligned} T\left(\frac{n}{2}\right) &= T\left(\frac{\frac{n}{2}}{2}\right) + c \\ &= T\left(\frac{n}{2^2}\right) + c \end{aligned}$$

$$T(n) = T\left(\frac{n}{2^2}\right) + 2c$$

$$T(n/2^2) = T(n/2^3) + c$$

$$T(n) = T(n/2^3) + 3c$$

.

.

.

$$T(n) = T(n/2^k) + kc$$

$$\text{suppose } n = 2^k \Rightarrow \log(n) = k$$

$$T(n) = T(n/n) + c \log(n)$$

$$T(n) = T(1) + c \log(n)$$

$$T(n) = c + c \log(n)$$

$$T(n) = c(\log(n) + 1)$$

$$T(n) = O(\log(n))$$

Binary Search – Analysis

- ▣ For an unsuccessful search:
 - The number of iterations in the loop is $\lfloor \log_2 n \rfloor + 1$
→ $O(\log_2 n)$
- ▣ For a successful search:
 - *Best-Case*: The number of iterations is 1. → $O(1)$
 - *Worst-Case*: The number of iterations is $\lfloor \log_2 n \rfloor + 1$ → $O(\log_2 n)$
 - *Average-Case*: The avg. # of iterations $< \log_2 n$ → $O(\log_2 n)$

0 1 2 3 4 5 6 7 ← an array with size 8

3 2 3 1 3 2 3 4 ← # of iterations

The average # of iterations = $21/8 < \log_2 8$

Growth-Rate Functions – Recursive Algorithms

```
void hanoi(int n, char source, char dest, char spare) { Cost
    if (n > 0) {
        c1
        hanoi(n-1, source, spare, dest);           c2
        cout << "Move top disk from pole " << source   c3
            << " to pole " << dest << endl;
        hanoi(n-1, spare, dest, source);           c4
    } }
```

- ▣ The time-complexity function $T(n)$ of a recursive algorithm is defined in terms of itself, and this is known as **recurrence equation** for $T(n)$.
- ▣ To find the growth-rate function for a recursive algorithm, we have to solve its recurrence relation.